



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 5: THIẾT KẾ XỬ LÝ (02)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1.** Hiểu và giải thích được vai trò của giao diện như một đặc tả xử lý.
2. **MT2.** Nhận diện và phân loại đầy đủ các sự kiện (events) phát sinh từ một màn hình giao diện.
3. **MT3.** Nắm vững các kỹ thuật xử lý trên giao diện: validation, state management, event handling, error handling.
4. **MT4.** Áp dụng được quy trình thiết kế xử lý từ giao diện đã có: từ phân tích màn hình → xác định sự kiện → thiết kế xử lý → cập nhật UML.
5. **MT5.** Đọc được một màn hình Figma/mockup và dịch thành Sequence Diagram + Class Diagram tương ứng.

NỘI DUNG

1. Các kỹ thuật xử lý trên giao diện

- a) Giao diện như một đặc tả xử lý
- b) Kỹ thuật 1: Kiểm tra dữ liệu đầu vào
- c) Kỹ thuật 2: Quản lý trạng thái
- d) Kỹ thuật 3: Xử lý sự kiện
- e) Kỹ thuật 4: Xử lý lỗi

2. Thiết kế xử lý từ giao diện đã có

3. Thực hành

1. Các kỹ thuật xử lý trên giao diện

1.1. Giao diện như một đặc tả xử lý

Giao diện không chỉ là bản vẽ thẩm mỹ, mỗi thành phần UI đều ẩn chứa một yêu cầu xử lý cụ thể mà hệ thống phải đáp ứng.

Thành phần UI	Câu hỏi xử lý ẩn chứa	Ví dụ
Ô nhập liệu (Input field)	Dữ liệu nhập vào cần được kiểm tra gì? Lỗi hiển thị như thế nào?	Email: kiểm tra định dạng, kiểm tra tồn tại trong hệ thống
Nút bấm (Button)	Khi bấm, hành động gì được kích hoạt? Điều kiện để nút được bấm?	Nút 'Đặt hàng' chỉ active khi giỏ hàng không rỗng
Danh sách thả xuống (Dropdown)	Dữ liệu danh sách lấy từ đâu? Chọn xong thì gì thay đổi?	Chọn tỉnh → cập nhật danh sách quận/huyện tương ứng

Email

Enter a valid email

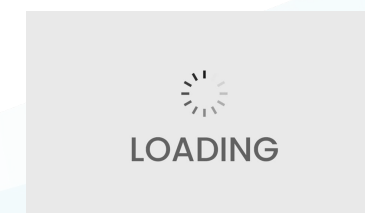
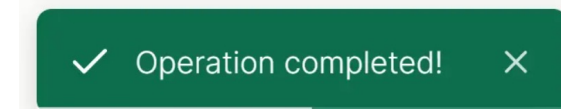
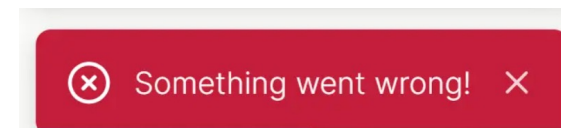
SUBMIT SUBMIT

1. Các kỹ thuật xử lý trên giao diện

1.1. Giao diện như một đặc tả xử lý

Giao diện không chỉ là bản vẽ thẩm mỹ, mỗi thành phần UI đều ẩn chứa một yêu cầu xử lý cụ thể mà hệ thống phải đáp ứng.

Thành phần UI	Câu hỏi xử lý ẩn chứa	Ví dụ
Bảng dữ liệu (Table/List)	Dữ liệu được tải khi nào? Sắp xếp, lọc được xử lý ở đâu?	Bảng đơn hàng: phân trang, lọc theo trạng thái, sắp xếp theo ngày
Thông báo (Alert/Toast)	Điều kiện nào kích hoạt? Bao lâu thì tự ẩn? Người dùng có thể đóng?	Toast 'Lưu thành công' xuất hiện 3 giây sau khi API trả về 200
Trạng thái loading	Khi nào hiển thị? Người dùng có thể làm gì trong lúc loading?	Spinner hiện khi gọi API, ẩn mọi nút bấm khác



1. Các kỹ thuật xử lý trên giao diện

1.2. Kỹ thuật 1: Validation (Kiểm tra dữ liệu đầu vào)

- ❑ Validation là quá trình kiểm tra xem dữ liệu mà người dùng nhập vào có đáp ứng các ràng buộc đã định nghĩa hay không, trước khi dữ liệu đó được xử lý hoặc lưu trữ.
- ❑ Validation không chỉ là kiểm tra hình thức (format) mà còn bao gồm kiểm tra tính hợp lệ theo nghiệp vụ (business rules).

1. Các kỹ thuật xử lý trên giao diện

1.2. Kỹ thuật 1: Validation (Kiểm tra dữ liệu đầu vào)

❑ Có 2 tầng validation quan trọng

Tầng validation	Vị trí xử lý	Ưu điểm	Hạn chế
Client-side validation	Trình duyệt / ứng dụng phía người dùng	Phản hồi tức thì (real-time), giảm số request lên server, trải nghiệm người dùng tốt hơn.	Có thể bị vô hiệu hoá hoặc bypass. KHÔNG thể tin tưởng hoàn toàn về bảo mật.
Server-side validation	Backend / API / Business Layer	Không thể bị bypass. Là tầng bảo vệ duy nhất đáng tin cậy về bảo mật và tính toàn vẹn dữ liệu.	Chỉ có kết quả sau khi gửi request, người dùng phải chờ.

1. Các kỹ thuật xử lý trên giao diện

1.2. Kỹ thuật 1: Validation (Kiểm tra dữ liệu đầu vào)

❑ Các loại validation thường dùng

- ✓ **Required validation:** kiểm tra trường bắt buộc không được để trống. **Ví dụ:** họ tên, email, mật khẩu.
- ✓ **Format validation:** kiểm tra định dạng dữ liệu. **Ví dụ:** email phải có @, số điện thoại phải đủ 10–11 chữ số, ngày tháng đúng định dạng dd/mm/yyyy.
- ✓ **Range validation:** kiểm tra giá trị nằm trong khoảng hợp lệ. b: tuổi từ 1–120, số lượng đặt hàng tối thiểu 1 tối đa 999.

1. Các kỹ thuật xử lý trên giao diện

1.2. Kỹ thuật 1: Validation (Kiểm tra dữ liệu đầu vào)

❑ Các loại validation thường dùng

- ✓ **Uniqueness validation:** kiểm tra giá trị chưa tồn tại trong hệ thống. **Ví dụ:** username chưa được đăng ký, mã sản phẩm chưa trùng. Loại này bắt buộc phải thực hiện ở server-side vì cần truy vấn database.
- ✓ **Business rule validation:** kiểm tra theo quy tắc nghiệp vụ. **Ví dụ:** ngày trả sách không được trước ngày mượn; tổng tiền đặt cọc không vượt quá giá trị đơn hàng.
- ✓ **Cross-field validation:** kiểm tra quan hệ giữa nhiều trường. **Ví dụ:** 'xác nhận mật khẩu' phải trùng với 'mật khẩu'; ngày kết thúc phải sau ngày bắt đầu.

1. Các kỹ thuật xử lý trên giao diện

1.2. Kỹ thuật 1: Validation (Kiểm tra dữ liệu đầu vào)

❑ Biểu diễn validation trong thiết kế xử lý

- ✓ Trong Sequence Diagram, validation thường được biểu diễn qua combined fragment alt với guard condition là kết quả validate.
- ✓ Trong Class Diagram, logic validate là một phương thức riêng trong lớp Service (không đặt trong Controller hay Entity).

1. Các kỹ thuật xử lý trên giao diện

1.2. Kỹ thuật 1: Validation (Kiểm tra dữ liệu đầu vào)

❑ Ví dụ:

❑ Màn hình 'Mượn sách':

→ Required: mã thẻ thư viện và mã sách không được trống

→ Business rule: thành viên không được mượn quá 5 quyển cùng lúc

→ Business rule: sách phải có trạng thái 'Có sẵn' (Available)

→ Cross-field: ngày trả dự kiến phải sau ngày mượn và không quá 30 ngày

→ Uniqueness: kiểm tra sách đó chưa đang được mượn bởi người dùng này

1. Các kỹ thuật xử lý trên giao diện

1.3. Kỹ thuật 2: State Management (Quản lý trạng thái)

- ❑ State là tập hợp tất cả dữ liệu và trạng thái hiển thị của giao diện tại một thời điểm nhất định.
- ❑ State xác định giao diện trông như thế nào và người dùng có thể làm gì.
- ❑ Khi state thay đổi, giao diện phải cập nhật tương ứng để phản ánh trạng thái mới.
- ❑ Mọi màn hình trong ứng dụng đều trải qua một vòng đời trạng thái. Hiểu và thiết kế đầy đủ các trạng thái này là yêu cầu bắt buộc để hệ thống hoạt động đúng đắn và người dùng không bị bối rối

1. Các kỹ thuật xử lý trên giao diện

1.3. Kỹ thuật 2: State Management (Quản lý trạng thái)

Trạng thái	Mô tả	Giao diện thể hiện	Ví dụ
Idle (Rảnh)	Màn hình sẵn sàng, chưa có hành động nào.	Form trống hoặc hiển thị dữ liệu ban đầu. Tất cả nút bấm hoạt động.	Màn hình đăng nhập khi vừa mở.
Loading (Đang tải)	Đang chờ kết quả từ server (API call đang chạy).	Spinner/skeleton hiển thị. Nút bấm bị disable để tránh gọi API nhiều lần.	Sau khi bấm 'Đăng nhập', chờ server xác thực.
Success (Thành công)	Xử lý hoàn tất, kết quả tích cực.	Hiển thị thông báo thành công (toast/alert). Cập nhật dữ liệu trên màn hình.	Đặt hàng thành công, hiển thị mã đơn hàng.
Error (Lỗi)	Xử lý thất bại hoặc validation không qua.	Hiển thị thông báo lỗi cụ thể. Form vẫn giữ dữ liệu đã nhập.	Lỗi 'Email đã tồn tại' khi đăng ký.
Empty (Trống)	Không có dữ liệu để hiển thị.	Màn hình trống với hướng dẫn hành động tiếp theo.	Giỏ hàng trống, lịch sử đơn hàng chưa có giao dịch.
Partial/Stale	Dữ liệu đã cũ, đang làm mới.	Hiển thị dữ liệu cũ kèm indicator 'đang cập nhật'.	Pull-to-refresh trên mobile.

1. Các kỹ thuật xử lý trên giao diện

1.3. Kỹ thuật 2: State Management (Quản lý trạng thái)

Các trạng thái không chuyển đổi tùy tiện. Chúng tuân theo một state machine (mô hình chuyển trạng thái) xác định: từ trạng thái nào có thể chuyển sang trạng thái nào, và điều kiện/sự kiện nào kích hoạt sự chuyển đổi đó.

1. Các kỹ thuật xử lý trên giao diện

1.3. Kỹ thuật 2: State Management (Quản lý trạng thái)

Ví dụ state machine cho form đặt hàng

Idle [bấm 'Đặt hàng'] → Loading

Loading [API trả về 200] → Success

Loading [API trả về 4xx/5xx] → Error

Loading [network timeout] → Error

Error [bấm 'Thử lại'] → Loading

Error [sửa form] → Idle

Success [tự động sau 3s] → Idle (reset form) hoặc Redirect

→ Thiết kế xử lý phải xử lý TẤT CẢ các nhánh chuyển trạng thái, không chỉ happy path.

1. Các kỹ thuật xử lý trên giao diện

1.4. Kỹ thuật 3: Event Handling (Xử lý sự kiện)

- ❑ Event là một thông điệp được phát sinh khi người dùng thực hiện một hành động trên giao diện hoặc khi hệ thống tự động kích hoạt một điều kiện.
- ❑ Mỗi event mang thông tin về loại hành động và đối tượng liên quan.
- ❑ Hệ thống phải có Event Handler (là một hàm xử lý) để phản hồi mỗi event.
- ❑ Phân loại event theo nguồn gốc:

1. Các kỹ thuật xử lý trên giao diện

1.4. Kỹ thuật 3: Event Handling (Xử lý sự kiện)

Loại event	Ví dụ	Xử lý điển hình
User action events	onClick (bấm nút), onChange (thay đổi ô nhập), onSubmit (gửi form), onFocus / onBlur (vào/rời ô nhập)	Validate dữ liệu, gọi API, cập nhật state, điều hướng màn hình.
Lifecycle events	onLoad (màn hình vừa mở), onUnmount (rời khỏi màn hình), onResume (quay lại màn hình)	Tải dữ liệu ban đầu, giải phóng tài nguyên, kiểm tra session.
System events	onNetworkChange (mạng thay đổi), onTimerTick (hết thời gian), onPush (nhận thông báo đẩy)	Cập nhật UI khi mất mạng, tự động đăng xuất, hiển thị badge thông báo.
Data events	onDataChange (dữ liệu thay đổi từ nguồn khác), onRefresh (cập nhật thủ công)	Real-time updates, làm mới danh sách, đồng bộ dữ liệu.

1. Các kỹ thuật xử lý trên giao diện

1.4. Kỹ thuật 3: Event Handling (Xử lý sự kiện)

Hai kỹ thuật tối ưu hoá Event Handling:

- ❑ **Debounce**

- ❑ **Throttle**

1. Các kỹ thuật xử lý trên giao diện

1.4. Kỹ thuật 3: Event Handling (Xử lý sự kiện)

❑ Debounce:

- ✓ Là kỹ thuật trì hoãn việc thực thi một hàm cho đến khi người dùng ngừng kích hoạt event trong một khoảng thời gian nhất định.
- ✓ Nếu event tiếp tục được kích hoạt liên tục, timer sẽ được reset. Hàm chỉ thực sự chạy sau khi event 'yên lặng' đủ lâu.
- ✓ Dùng khi không cần phản hồi mỗi lần người dùng gõ phím, mà chỉ cần phản hồi khi họ tạm dừng. Ví dụ: thanh tìm kiếm → chỉ gọi API sau khi người dùng ngừng gõ 300–500ms, không gọi API sau mỗi ký tự.
- ✓ Lợi ích: giảm đáng kể số lần gọi API, giảm tải server, tránh hiển thị kết quả tạm thời không cần thiết.

1. Các kỹ thuật xử lý trên giao diện

1.4. Kỹ thuật 3: Event Handling (Xử lý sự kiện)

❑ Throttle:

- ✓ Là kỹ thuật giới hạn số lần một hàm được thực thi trong một khoảng thời gian nhất định.
- ✓ Dù event được kích hoạt bao nhiêu lần, hàm cũng chỉ chạy tối đa một lần trong mỗi khoảng thời gian đã định.
- ✓ Dùng khi cần phản hồi liên tục nhưng có kiểm soát tần suất. Ví dụ: xử lý scroll event (tính toán vị trí cuộn), infinite scroll (tải thêm dữ liệu khi cuộn gần cuối trang).
- ✓ Debounce đợi sự kiện kết thúc rồi mới thực thi (1 lần duy nhất). Throttle thực thi đều đặn theo chu kỳ trong suốt thời gian sự kiện diễn ra.

1. Các kỹ thuật xử lý trên giao diện

1.5. Kỹ thuật 4: Error Handling (Xử lý lỗi)

Error Handling là tập hợp các chiến lược và cơ chế để phát hiện, phân loại và phản hồi các tình huống lỗi xảy ra trong quá trình tương tác giữa người dùng và hệ thống, theo cách vừa thông báo đầy đủ cho người dùng vừa không làm gián đoạn trải nghiệm một cách không cần thiết.

1. Các kỹ thuật xử lý trên giao diện

1.5. Kỹ thuật 4: Error Handling (Xử lý lỗi)

Loại lỗi	Nguồn gốc	Hiển thị phù hợp	Ví dụ
Validation error	Dữ liệu nhập không hợp lệ	Inline error dưới trường nhập liệu tương ứng → người dùng biết ngay cần sửa gì.	'Email không đúng định dạng' xuất hiện dưới ô email.
Business logic error	Vi phạm quy tắc nghiệp vụ	Alert hoặc inline message ở đầu form → lỗi cần người dùng đọc kỹ để hiểu.	'Bạn đã mượn đủ 5 quyển sách, không thể mượn thêm.'
Network error	Mất kết nối, timeout	Toast hoặc banner với nút 'Thử lại' → lỗi tạm thời, người dùng cần hành động.	'Không có kết nối mạng. Kiểm tra và thử lại.'
Server error (5xx)	Lỗi phía server	Modal hoặc trang lỗi riêng → lỗi nghiêm trọng cần dừng luồng hiện tại.	'Hệ thống đang gặp sự cố. Vui lòng thử lại sau.'
Session expired	Phiên đăng nhập hết hạn	Redirect về trang đăng nhập kèm thông báo → bắt buộc phải xác thực lại.	'Phiên làm việc đã hết hạn. Vui lòng đăng nhập lại.'

1. Các kỹ thuật xử lý trên giao diện

1.5. Kỹ thuật 4: Error Handling (Xử lý lỗi)

Nguyên tắc Error Handling tốt

- Thông báo lỗi phải cụ thể: nói rõ lỗi gì, ở đâu, và người dùng cần làm gì tiếp theo.
- Không hiển thị mã lỗi kỹ thuật (stack trace, HTTP 500) cho người dùng cuối.
- Giữ lại dữ liệu đã nhập: form không được reset khi có lỗi → người dùng không phải nhập lại từ đầu.
- Phân biệt lỗi người dùng (user error) và lỗi hệ thống (system error): cách xử lý khác nhau.
- Luôn có fallback: nếu mọi thứ đều thất bại, vẫn phải cho người dùng biết và thoát ra được.

2. Thiết kế xử lý giao diện đã có

2.1. Tổng quan

- ❑ Thiết kế xử lý từ giao diện là quá trình đọc một màn hình đã được thiết kế (wireframe, mockup, hoặc giao diện thực) và dịch nó thành các bản vẽ kỹ thuật (Sequence Diagram, Class Diagram) mô tả đầy đủ những gì hệ thống phải làm để giao diện đó hoạt động đúng.

2. Thiết kế xử lý giao diện đã có

2.1. Tổng quan

Là quá trình phân tích có hệ thống một màn hình giao diện để:

- (1) xác định tất cả các sự kiện có thể phát sinh,
 - (2) thiết kế luồng xử lý cho từng sự kiện bao gồm các bước xử lý, dữ liệu cần thiết và kết quả trả về,
 - (3) ánh xạ các luồng xử lý này thành Sequence Diagram, và
 - (4) cập nhật Class Diagram với các phương thức và lớp mới cần bổ sung.
- Đây là bước cầu nối trực tiếp từ thiết kế giao diện sang cài đặt code.

2. Thiết kế xử lý giao diện đã có

2.1. Tổng quan

Bước	Tên bước	Nội dung thực hiện	Đầu ra
1	Phân tích màn hình	Đọc kỹ giao diện, nhận diện tất cả thành phần UI: input fields, buttons, labels, tables, dropdowns, alerts.	Danh sách thành phần UI.
2	Xác định sự kiện	Với mỗi thành phần UI, liệt kê TẤT CẢ events có thể xảy ra (cả happy path và error path).	Bảng sự kiện đầy đủ.
3	Thiết kế xử lý	Với mỗi event quan trọng, thiết kế luồng xử lý: validate gì → gọi lớp nào → xử lý gì → kết quả gì.	Pseudocode / flow cho từng event.
4	Vẽ Sequence Diagram	Chuyển luồng xử lý ở bước 3 thành Sequence Diagram UML với đầy đủ thành phần đã học.	Sequence Diagram cho từng luồng.
5	Cập nhật Class Diagram	Từ các thông điệp trong Sequence Diagram, bổ sung phương thức còn thiếu vào Class Diagram.	Class Diagram hoàn chỉnh.

2. Thiết kế xử lý giao diện đã có

2.2. Bước 1 & 2: Phân tích màn hình và xác định sự kiện

- ❑ Đây là bước đòi hỏi sự tỉ mỉ nhất.
- ❑ Nhiều người chỉ nghĩ đến sự kiện bấm nút và bỏ qua rất nhiều sự kiện ngầm.
- ❑ Hãy nhìn vào mọi thành phần UI và đặt câu hỏi: 'Có điều gì xảy ra với thành phần này không?'

2. Thiết kế xử lý giao diện đã có

2.2. Bước 1 & 2: Phân tích màn hình và xác định sự kiện

Gợi ý: Danh sách kiểm tra sự kiện cho mỗi màn hình

- Khi màn hình vừa mở (onLoad): dữ liệu nào cần tải? Điều kiện hiển thị?
- Mỗi ô nhập liệu: onFocus (focus vào), onChange (thay đổi), onBlur (rời khỏi).
- Mỗi nút bấm chính: điều kiện để active? onClick → làm gì?
- Dropdown / select: onSelect → ảnh hưởng đến trường nào khác?
- Danh sách / bảng: onScroll đến cuối → tải thêm? onClick row → chi tiết?
- Trường hợp lỗi: validation thất bại → hiển thị gì? API lỗi → làm gì?
- Hủy / thoát: người dùng thoát giữa chừng → có cảnh báo mất dữ liệu không?

2. Thiết kế xử lý giao diện đã có

2.3. Bước 3: Thiết kế xử lý bằng pseudocode

- ❑ Trước khi vẽ Sequence Diagram, nên viết pseudocode để tư duy rõ ràng về luồng xử lý.
- ❑ Pseudocode không cần đúng cú pháp ngôn ngữ lập trình, quan trọng là thể hiện đúng logic và thứ tự xử lý.
- ❑ Pseudocode là mô tả thuật toán bằng ngôn ngữ tự nhiên kết hợp với các cấu trúc điều khiển cơ bản (if-else, loop, call), dùng để phác thảo logic xử lý của một chức năng trước khi cài đặt bằng ngôn ngữ lập trình cụ thể.
- ❑ Pseudocode đóng vai trò trung gian giữa mô tả bằng lời và Sequence Diagram.

2. Thiết kế xử lý giao diện đã có

2.4. Bước 4 & 5: Dịch pseudocode sang UML

- ❑ Sau khi pseudocode đã rõ ràng, việc dịch sang Sequence Diagram và Class Diagram trở nên trực tiếp và có hệ thống.
- ❑ Đây là bảng ánh xạ giữa cấu trúc pseudocode và ký hiệu UML:

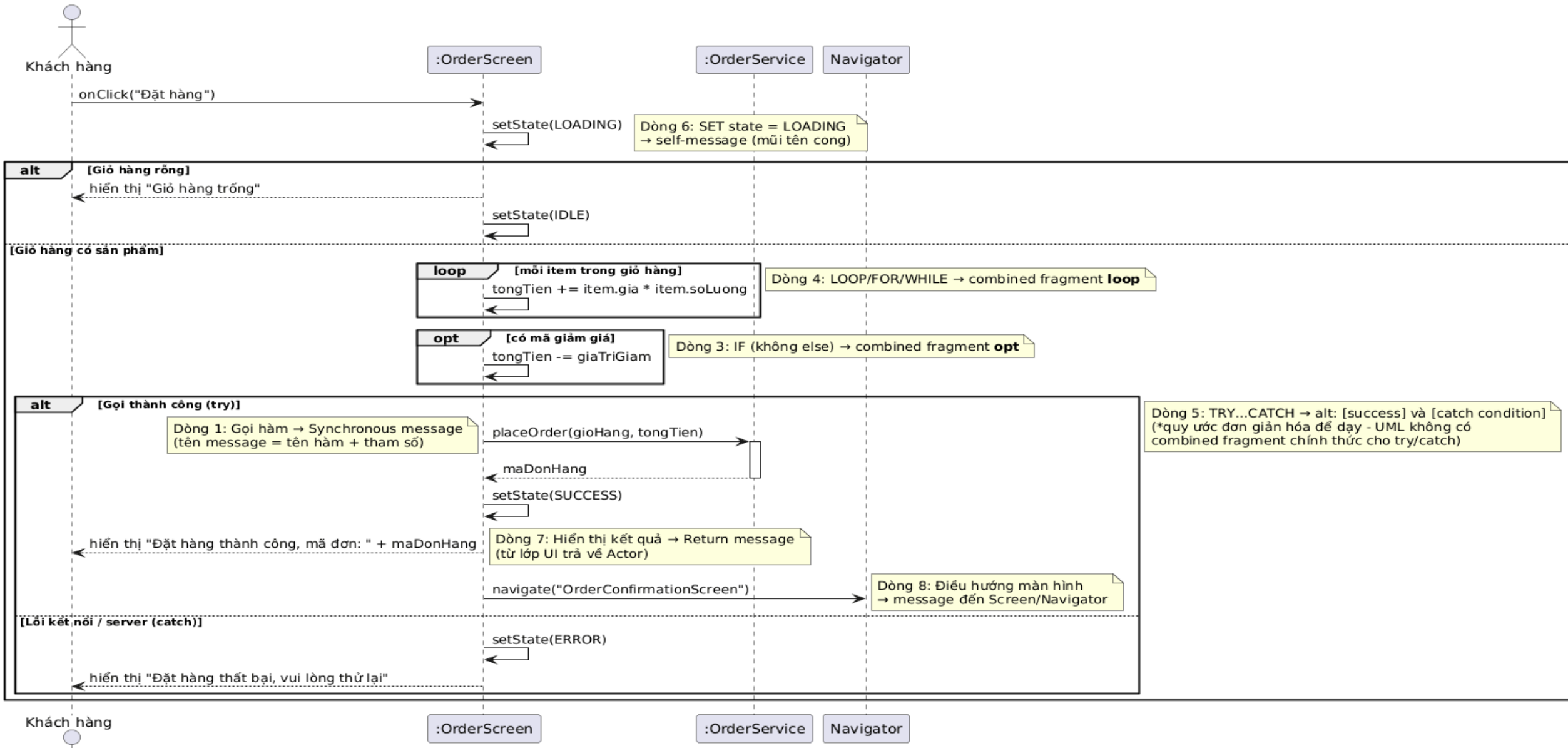
2. Thiết kế xử lý giao diện đã có

2.4. Bước 4 & 5: Dịch pseudocode sang UML

Cấu trúc trong Pseudocode	Tương ứng trong UML	Ghi chú
Gọi hàm: <code>service.placeOrder(...)</code>	Synchronous message →	Tên message = tên hàm + tham số
IF ... ELSE ...	Combined fragment: alt	Mỗi nhánh là một operand với guard condition
IF ... (không có else)	Combined fragment: opt	Guard condition là điều kiện của IF
LOOP / FOR / WHILE	Combined fragment: loop	Ghi điều kiện lặp trong guard
TRY ... CATCH	alt: [success] và [catch condition]	Mỗi loại exception là một nhánh alt
SET state = LOADING	Message nội bộ (self-message)	Mũi tên cong quay lại chính lifeline đó
Hiển thị kết quả cho người dùng	Return message ←	Từ lớp UI trả về Actor
Điều hướng màn hình	Message đến Screen/Navigator	Là message cuối cùng trước khi kết thúc

2. Thiết kế xử lý giao diện đã có

Đặt hàng (Place Order))



2. Thiết kế xử lý giao diện đã có

2.5. Lưu ý về thiết kế lặp (Iterative Design)

- ❑ Thiết kế xử lý không phải là một quy trình tuyến tính một chiều.
- ❑ Giao diện thiếu trạng thái: màn hình chưa thiết kế trạng thái loading hay empty → cần bổ sung vào thiết kế giao diện.
- ❑ Class Diagram chưa đủ phương thức: các lớp ở Chương 3 chỉ có thuộc tính, chưa có phương thức → cần bổ sung từ Sequence Diagram.
- ❑ ERD có thể chưa chính xác: khi thiết kế xử lý, đôi khi phát hiện cần thêm bảng hoặc cột mà ERD chưa có.
- ❑ Use Case chưa đủ chi tiết: Sequence Diagram buộc phải rõ ràng hơn Use Case, nên sẽ bộc lộ những khoảng trống trong mô tả yêu cầu.

THỰC HÀNH